

Assignment no.4

Title: Implement a train route management system using a doubly linked list to store stations. It should support: a. Insert a station b. Delete a station c. Display all stations d. Search a station e. Print stations in reverse order

Name: Diya Ravindra Bhansali

PRN: 125B1F271

Class: SY BTech IT

Source Code:

```
/*  
Assignment No: 04  
Name:Diya Ravindra Bhansali  
PRN:125B1F271  
Class & Div : SY BTech IT A4  
*/  
  
#include <bits/stdc++.h>  
using namespace std;  
class Station  
{  
public:  
    string st_name;  
    Station *next,*prev;  
  
    Station()  
    {  
        next=NULL;  
        prev=NULL;  
    }  
}
```

```
void get_st_details()
{
    cout<<"Enter Station name: ";
    cin>>st_name;
}

void print_st_details()
{
    cout<<"\nStation: "<<st_name;
}
};

class train
{
public:
    string t_name;
    string t_code;
    Station *head,*tail;

    train()
    {
        head=tail=NULL;
    }

    void get_train_details()
    {
        cout<<"\nEnter Train name:";
```

```
    cin>>t_name;
    cout<<"\nEnter the train code:";
    cin>>t_code;
}
```

```
void insert_st()
{
    Station *newStation = new Station();
    newStation->get_st_details();
    if (head == NULL)
    {
        head = newStation;
        tail = newStation;
    }
    else
    {
        tail->next = newStation;
        newStation->prev=tail;
        tail = newStation;
    }
}
```

```
void remove_st(string sname)
{
    Station *curr = head;
    while (curr != NULL && curr->st_name != sname)
    {
```

```

    curr = curr->next;
}
if (curr == NULL)
{
    cout << "\nStation Not Found!\n";
    return;
}
if (curr == head)
{
    head = curr->next;
    if (head != NULL)
        head->prev = NULL;
    else
        tail = NULL;
    delete curr;
    cout << "\nStation Removed Successfully!\n";
    return;
}
if (curr == tail)
{
    tail = curr->prev;
    tail->next = NULL;
    delete curr;
    cout << "\nStation Removed Successfully!\n";
    return;
}
curr->prev->next = curr->next;

```

```

curr->next->prev = curr->prev;
delete curr;
cout << "\nStation Removed Successfully!\n";
}

void display_st()
{
    Station *curr = head;
    cout << "\n\n**** Train Route ****\n";
    if (head == NULL)
    {
        cout << "HEAD -> NULL";
        return;
    }
    while (curr != NULL)
    {
        cout << curr->st_name;
        if (curr->next != NULL)
            cout << " -> ";
        curr = curr->next;
    }
    cout << " -> NULL";
}

void rev_display_st()
{
    Station *curr = tail;
    cout << "\n\n**** Reverse Train Route ****\n";

```

```
if (tail == NULL)
{
    cout << "NULL";
    return;
}
while (curr != NULL)
{
    cout << curr->st_name;
    if (curr->prev != NULL)
        cout << " -> ";
    curr = curr->prev;
}
cout << " -> NULL";
}
```

```
void search_st(string sname)
{
    Station *curr = head;
    while (curr != NULL)
    {
        if (curr->st_name == sname)
        {
            cout << "\n\n*** Station Info ***";
            curr->print_st_details();
            return;
        }
        curr = curr->next;
    }
}
```

```

    }
    cout << "\nStation Not Found!\n";
}
};

```

```

int main()
{
    int ch;
    train *t1=new train;
    do{
        cout << "\n===== ";
        cout << "\n  TRAIN ROUTE MANAGEMENT SYSTEM";
        cout << "\n    Using Linked List";
        cout << "\n===== ";
        cout << "\n1. Insert Station";
        cout << "\n2. Remove Station";
        cout << "\n3. Display Route";
        cout << "\n4. Reverse Display Route";
        cout << "\n5. Search Station";
        cout << "\n6. Exit";
        cout << "\n\nEnter your Choice: ";
        cin >> ch;
        switch (ch)
        {
            case 1:
            {
                t1->get_train_details();

```

```
char choice;
do
{
    t1->insert_st();
    cout<<"\nDo you want to add more station(Y/N):";
    cin>>choice;
}while(choice!='N');
break;
}
case 2:
{
    string sname;
    cout<<"\nEnter the station name you want to remove:";
    cin>>sname;
    t1->remove_st(sname);
    break;
}
case 3:
    t1->display_st();
    break;
case 4:
    t1->rev_display_st();
    break;
case 5:
{
    string sname;
    cout<<"\nEnter the station name you want to search:";
```

```

        cin>>sname;
        t1->search_st(sname);
        break;
    }
    case 6:
        exit(0);
    default:
        cout << "\nInvalid Choice!!\n";
    }
}while(ch!=6);
return 0;
}

```

Output:

```

/*
=====

  TRAIN ROUTE MANAGEMENT SYSTEM
    Using Linked List
=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

```

Enter your Choice: 1

Enter Train name:local

Enter the train code:5654

Enter Station name: akurdi

Do you want to add more station(Y/N):y

Enter Station name: kasarwadi

Do you want to add more station(Y/N):y

Enter Station name: khadki

Do you want to add more station(Y/N):y

Enter Station name: shivajinagar

Do you want to add more station(Y/N):y

Enter Station name: pune

Do you want to add more station(Y/N):N

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route

4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 3

**** Train Route ****

akurdi -> kasarwadi -> khadki -> shivajinagar -> pune -> NULL

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 4

**** Reverse Train Route ****

pune -> shivajinagar -> khadki -> kasarwadi -> akurdi -> NULL

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 2

Enter the station name you want to remove:khadki

Station Removed Successfully!

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 3

**** Train Route ****

akurdi -> kasarwadi -> shivajinagar -> pune -> NULL

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 5

Enter the station name you want to search:pune

*** Station Info ***

Station: pune

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station

2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 5

Enter the station name you want to search:khadki

Station Not Found!

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 3

**** Train Route ****

akurdi -> kasarwadi -> shivajinagar -> pune -> NULL

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 4

**** Reverse Train Route ****

pune -> shivajinagar -> kasarwadi -> akurdi -> NULL

=====

TRAIN ROUTE MANAGEMENT SYSTEM

Using Linked List

=====

1. Insert Station
2. Remove Station
3. Display Route
4. Reverse Display Route
5. Search Station
6. Exit

Enter your Choice: 6

Process returned 0 (0x0) execution time : 133.002 s

Press any key to continue.

*/

Code Analysis Report

Train Route Management System (Doubly Linked List Implementation)

Field	Detail
Student	Diya Ravindra Bhansali
PRN	125B1F271
Class	SY B.Tech IT, A4
Assignment No.	04
Language / Paradigm	C++ (Object-Oriented Programming)
Core Data Structure	Doubly Linked List (custom-implemented)

1. Overview

The program models a train's route as a doubly linked list, where each node (Station) represents a stop and carries next/prev pointers. A single train object owns head and tail pointers into this list and exposes operations to insert a station at the tail, remove a station by name, display the route forward and in reverse, and search for a station. A menu-driven main() loop drives these operations interactively.

Overall the design correctly applies the doubly linked list pattern taught in the DSA course — tail-pointer insertion, prev-pointer relinking on deletion, and O(1) reverse traversal using the tail pointer instead of recursion or a second pass. The sample run in the trailing comment block confirms insert, remove, display, reverse-display, and search all behave as expected.

2. Structural Analysis

2.1 Classes

- Station — a plain node holding st_name plus next and prev pointers, with its own input/output helper methods.
- train — the list wrapper holding head, tail, and train metadata (t_name, t_code), with all list operations as member functions.

This separation (node class vs. list-manager class) is good OOP practice and keeps node concerns (data, printing) separate from list concerns (traversal, linking).

2.2 Time Complexity

Operation	Complexity	Notes
insert_st (tail insert)	$O(1)$	Direct append via the tail pointer — no traversal needed.
remove_st (by name)	$O(n)$	Linear search to locate the node; $O(1)$ relink once found.
display_st	$O(n)$	Single forward pass from head.
rev_display_st	$O(n)$	Single backward pass from tail — better than reversing a singly linked list.
search_st	$O(n)$	Linear scan; no index or hash lookup.

Space complexity is $O(n)$ for n stations, with a small constant overhead per node for the extra prev pointer relative to a singly linked list.

3. Issues Identified

3.1 Memory Management

- No destructor in Station or train: removed nodes are freed correctly inside `remove_st`, but the remaining chain (and the train object `t1` itself, allocated with `new` in main) is never released. On a short-lived console program this only leaks at process exit, but the habit doesn't scale to long-running or repeated-allocation code.
- Raw owning pointers throughout, with no copy constructor / copy-assignment / move semantics defined for either class (violation of the Rule of Three/Five). Copying a train object today would shallow-copy head/tail, leaving two objects pointing at the same nodes — a double-free waiting to happen if such a copy were ever made.

3.2 Input Handling

- Station names are read with `cin >> st_name`, so multi-word names (e.g. "Pune Junction") silently truncate at the first space.
- The menu choice (`cin >> ch`) is never validated — a non-numeric entry puts `cin` into a fail state, and the program either loops forever on the same prompt or misbehaves rather than reporting invalid input.
- No `cin.clear()` / `cin.ignore()` guard anywhere, so a single bad token can cascade into repeated failed reads.

3.3 Design / Scalability

- Only one train (`t1`) can exist at a time; choosing "Insert Station" again re-prompts for a train name/code but keeps appending to the same list, so the system cannot actually manage multiple distinct train routes despite the class being named `train`.
- `search_st` and `remove_st` both do a case-sensitive linear scan; "Pune" and "pune" are treated as different stations, and there is no way to search by position or partial match.

- `insert_st` never checks for a duplicate station name, so the same stop can be added twice.
- using namespace `std`; is used at file scope, which is fine for a small assignment but is generally discouraged as programs grow, since it pulls the entire `std` namespace into global scope.
- `remove_st` repeats the same "Station Removed Successfully" output and `delete curr`; logic three times (head case, tail case, middle case) — functionally correct but a candidate for consolidation.
- All member functions read/print via `cin/cout` directly rather than returning values, which tightly couples the data structure to console I/O and would make the class harder to reuse (e.g., in a GUI or file-backed version) without rewriting it.
- No persistence — all routes are lost when the program exits, since there is no file save/load.

4. Suggested Optimizations

4.1 Memory Safety

- Add a destructor to `train` that walks the list from head and deletes every node, and `delete t1` (or better, avoid heap-allocating it) at the end of main.
- Prefer `std::unique_ptr<Station>` for `next` (with a raw back-pointer for `prev`) or use `std::list<Station>` from the STL, which gives automatic cleanup and eliminates manual `delete` calls entirely.

4.2 Faster Search

- Maintain an `unordered_map<string, Station*>` alongside the linked list, updated on every insert/remove, to bring `search_st` and the lookup inside `remove_st` down to average $O(1)$ instead of $O(n)$.
- Alternatively, if the station count stays small (a handful of stops per route), the current $O(n)$ scan is perfectly reasonable and this optimization can be skipped in favor of simplicity.

4.3 Robust Input

- Read station names with `std::getline` after `cin.ignore()` so multi-word names are captured correctly.
- Wrap the menu read in a loop that checks `cin.fail()`, calls `cin.clear()` and `cin.ignore(numeric_limits<streamsize>::max(), '\n')`, and re-prompts on invalid input.

4.4 Multi-Train Support

- Replace the single `train *t1` with a `std::map<std::string, train>` keyed by train code (or a `std::vector<train>`), and add a "Select / Create Train" step in the menu so the system can genuinely manage more than one route at a time.

4.5 Code Quality

- Factor the three near-identical branches in `remove_st` into a single unlink step (relink `curr->prev` and `curr->next`, adjusting head/tail only when `curr` is one of them) followed by one shared `delete` and `print` statement.
- Mark read-only member functions (`display_st`, `rev_display_st`, `search_st`, `print_st_details`) as `const`, and mark parameters passed by value but not modified (e.g. `const string &sname`) as `const` references to avoid unnecessary copies.
- Add a case-insensitive comparison helper for `search_st` / `remove_st` if station names should match regardless of case.

- Optionally add simple file I/O (save the route to a .txt/.csv on exit, load it on startup) so routes persist across runs.

5. Conclusion

The implementation correctly demonstrates the core doubly linked list operations — tail insertion, bidirectional traversal, and safe node removal with pointer relinking — and the sample execution trace confirms all six menu operations work as intended. The main gaps are typical of a first pass at a DSA assignment rather than flaws in the underlying algorithm: missing destructors and Rule-of-Three compliance, unvalidated console input, single-word-only station names, and a design that supports only one train route per run. None of these affect correctness for the given test case, but addressing them — particularly the destructor, input validation, and multi-word station names — would make the program noticeably more robust without changing its core linked-list logic.